

CONTROL AND OBSERVABILITY

Observability with OpenTelemetry

Export traces, metrics, and events from the Agent SDK to your observability backend using OpenTelemetry.

When you run agents in production, you need visibility into what they did:

- which tools they called
- how long each model request took
- how many tokens were spent
- where failures occurred

The Agent SDK can export this data as OpenTelemetry traces, metrics, and log events to any backend that accepts the OpenTelemetry Protocol (OTLP), such as Honeycomb, Datadog, Grafana, Langfuse, or a self-hosted collector.

This guide explains how the SDK emits telemetry, how to configure the export, and how to tag and filter the data once it reaches your backend. To read token usage and cost directly from the SDK response stream instead of exporting to a backend, see [Track cost and usage](#).

How telemetry flows from the SDK

The Agent SDK runs the Claude Code CLI as a child process and communicates with it over a local pipe. The CLI has OpenTelemetry instrumentation built in: it records spans around each model request and tool execution, emits metrics for token and cost counters, and emits structured log events for prompts and tool results. The SDK does not produce telemetry of its own. Instead, it passes configuration through to the CLI process, and the CLI exports directly to your collector.

Configuration is passed as environment variables. By default, the child process inherits your application's environment, so you can configure telemetry in either of two places:

- **Process environment:** set the variables in your shell, container, or orchestrator before your application starts. Every `query()` call picks them up automatically with no code change. This is

the recommended approach for production deployments.

- **Per-call options:** set the variables in `ClaudeAgentOptions.env` (Python) or `options.env` (TypeScript). Use this when different agents in the same process need different telemetry settings. In Python, `env` is merged on top of the inherited environment. In TypeScript, `env` replaces the inherited environment entirely, so include `...process.env` in the object you pass.

The CLI exports three independent OpenTelemetry signals. Each has its own enable switch and its own exporter, so you can turn on only the ones you need.

Signal	What it contains	Enable with
Metrics	Counters for tokens, cost, sessions, lines of code, and tool decisions	<code>OTEL_METRICS_EXPORTER</code>
Log events	Structured records for each prompt, API request, API error, and tool result	<code>OTEL_LOGS_EXPORTER</code>
Traces	Spans for each interaction, model request, tool call, and hook (beta)	<code>OTEL_TRACES_EXPORTER</code> plus <code>CLAUDE_CODE_ENHANCED_TELEMETRY_BETA=1</code>

For the complete list of metric names, event names, and attributes, see the Claude Code [Monitoring](#) reference. The Agent SDK emits the same data because it runs the same CLI. Span names are listed in [Read agent traces](#) below.

Enable telemetry export

Telemetry is off until you set `CLAUDE_CODE_ENABLE_TELEMETRY=1` and choose at least one exporter. The most common configuration sends all three signals over OTLP HTTP to a collector.

The following example sets the variables in a dictionary and passes them through `options.env`. The agent runs a single task, and the CLI exports spans, metrics, and events to the collector at `collector.example.com` while the loop consumes the response stream:

```

import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

OTEL_ENV = {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    # Required for traces, which are in beta. Metrics and log events do not
    # need this.
    "CLAUDE_CODE_ENHANCED_TELEMETRY_BETA": "1",
    # Choose an exporter per signal. Use otlp for the SDK; see the Note below.
    "OTEL_TRACES_EXPORTER": "otlp",
    "OTEL_METRICS_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    # Standard OTLP transport configuration.
    "OTEL_EXPORTER_OTLP_PROTOCOL": "http/protobuf",
    "OTEL_EXPORTER_OTLP_ENDPOINT": "http://collector.example.com:4318",
    "OTEL_EXPORTER_OTLP_HEADERS": "Authorization=Bearer your-token",
}

async def main():
    options = ClaudeAgentOptions(env=OTEL_ENV)
    async for message in query(
        prompt="List the files in this directory", options=options
    ):
        print(message)

asyncio.run(main())

```

Because the child process inherits your application's environment by default, you can achieve the same result by exporting these variables in a Dockerfile, Kubernetes manifest, or shell profile and omitting `options.env` entirely.

❗ The `console` exporter writes telemetry to standard output, which the SDK uses as its message channel. Do not set `console` as an exporter value when running through the SDK. To inspect telemetry locally, point `OTEL_EXPORTER_OTLP_ENDPOINT` at a local collector or an all-in-one Jaeger container instead.

Flush telemetry from short-lived calls

The CLI batches telemetry and exports on an interval. On a clean process exit it attempts to flush

pending data, but the flush is bounded by a short timeout, so spans can still be dropped if the collector is slow to respond. If your process is killed before the CLI shuts down, anything still in the batch buffer is lost. Lowering the export intervals reduces both windows.

By default, metrics export every 60 seconds and traces and logs export every 5 seconds. The following example shortens all three intervals so that data reaches the collector while a short task is still running:

```
OTEL_ENV = {  
    # ... exporter configuration from the previous example ...  
    "OTEL_METRIC_EXPORT_INTERVAL": "1000",  
    "OTEL_LOGS_EXPORT_INTERVAL": "1000",  
    "OTEL_TRACES_EXPORT_INTERVAL": "1000",  
}
```

Read agent traces

Traces give you the most detailed view of an agent run. With

`CLAUDE_CODE_ENHANCED_TELEMETRY_BETA=1` set, each step of the agent loop becomes a span you can inspect in your tracing backend:

- `claude_code.interaction` : wraps a single turn of the agent loop, from receiving a prompt to producing a response.
- `claude_code.llm_request` : wraps each call to the Claude API, with model name, latency, and token counts as attributes.
- `claude_code.tool` : wraps each tool invocation, with child spans for the permission wait (`claude_code.tool.blocked_on_user`) and the execution itself (`claude_code.tool.execution`).
- `claude_code.hook` : wraps each hook execution. Requires detailed beta tracing (`ENABLE_BETA_TRACING_DETAILED=1` and `BETA_TRACING_ENDPOINT`) in addition to the variables above.

The `llm_request` , `tool` , and `hook` spans are children of the enclosing

`claude_code.interaction` span. When the agent spawns a subagent through the Task tool, the subagent's `llm_request` and `tool` spans nest under the parent agent's `claude_code.tool` span, so the full delegation chain appears as one trace.

Spans carry a `session.id` attribute by default. When you make several `query()` calls against the

same **session**, filter on `session.id` in your backend to see them as one timeline. The attribute is omitted if `OTEL_METRICS_INCLUDE_SESSION_ID` is set to a falsy value.

❗ Tracing is in beta. Span names and attributes may change between releases. See [Traces \(beta\)](#) in the Monitoring reference for the trace exporter configuration variables.

Link traces to your application

The SDK automatically propagates W3C trace context into the CLI subprocess. When you call `query()` while an OpenTelemetry span is active in your application, the SDK injects `TRACEPARENT` and `TRACESTATE` into the child process environment, and the CLI reads them so its `claude_code.interaction` span becomes a child of your span. The agent run then appears inside your application's trace instead of as a disconnected root.

When trace-context propagation is enabled, the CLI also forwards `TRACEPARENT` to every Bash and PowerShell command it runs. If a command launched through the Bash tool emits its own OpenTelemetry spans, those spans nest under the `claude_code.tool.execution` span that wraps the command.

Auto-injection is skipped when you set `TRACEPARENT` explicitly in `options.env`, so you can pin a specific parent context if needed. Interactive CLI sessions ignore inbound `TRACEPARENT` entirely; only Agent SDK and `claude -p` runs honor it. See [Traces \(beta\)](#) in the Monitoring reference for the full span and attribute reference.

Tag telemetry from your agent

By default, the CLI reports `service.name` as `claude-code`. If you run several agents, or run the SDK alongside other services that export to the same collector, override the service name and add resource attributes so you can filter by agent in your backend.

The following example renames the service and attaches deployment metadata. These values are applied as OpenTelemetry resource attributes on every span, metric, and event the agent emits:

```
options = ClaudeAgentOptions(
    env={
        # ... exporter configuration ...
        "OTEL_SERVICE_NAME": "support-triage-agent",
        "OTEL_RESOURCE_ATTRIBUTES":
            "service.version=1.4.0,deployment.environment=production",
    },
)
```

Attribute actions to your end users

The CLI attaches **identity attributes** to every event based on the credential it uses to call Anthropic. When you build an application that serves many end users from one deployment, these attributes identify your service's credential, not the end user on whose behalf the agent acted.

To make tool calls and MCP activity attributable to your application's end users, inject end-user identity as resource attributes on each `query()` call. Percent-encode values before interpolating them, since `OTEL_RESOURCE_ATTRIBUTES` **reserves commas, spaces, and equals signs**. The following example attaches the requesting user and tenant to every span and event from one request:

```
from urllib.parse import quote

options = ClaudeAgentOptions(
    env={
        # ... exporter configuration ...
        "OTEL_RESOURCE_ATTRIBUTES": f"enduser.id=
{quote(request.user_id)},tenant.id={quote(request.tenant_id)}",
    },
)
```

With end-user identity attached, the `tool_decision`, `tool_result`, `mcp_server_connection`, and `permission_mode_changed` events become a per-user audit trail you can forward to a Security Information and Event Management (SIEM) platform. See **Audit security events** in the Monitoring reference for the full list of security-relevant events and the attributes each one carries.

Control sensitive data in exports

Telemetry is structural by default. Durations, model names, and tool names are recorded on every

span; token counts are recorded when the underlying API request returns usage data, so spans for failed or aborted requests may omit them. The content your agent reads and writes is not recorded by default. These opt-in variables add content to the exported data:

Variable	Adds
<code>OTEL_LOG_USER_PROMPT</code> <code>TS=1</code>	Prompt text on <code>claude_code.user_prompt</code> events and on the <code>claude_code.interaction</code> span
<code>OTEL_LOG_TOOL_DETAILED</code> <code>LS=1</code>	Tool input arguments (file paths, shell commands, search patterns) on <code>claude_code.tool_result</code> events
<code>OTEL_LOG_TOOL_CONTENT</code> <code>NT=1</code>	Full tool input and output bodies as span events on <code>claude_code.tool</code> , truncated at 60 KB. Requires <u>tracing</u> to be enabled
<code>OTEL_LOG_RAW_API_BODY</code> <code>DIES</code>	Full Anthropic Messages API request and response JSON as <code>claude_code.api_request_body</code> and <code>claude_code.api_response_body</code> log events. Set to <code>1</code> for inline bodies truncated at 60 KB, or <code>file:<dir></code> for untruncated bodies on disk with a <code>body_ref</code> path in the event. Bodies include the entire conversation history and have extended-thinking content redacted. Enabling this implies consent to everything the three variables above would reveal

Leave these unset unless your observability pipeline is approved to store the data your agent handles. See Security and privacy in the Monitoring reference for the full list of attributes and redaction behavior.

Related documentation

These guides cover adjacent topics for monitoring and deploying agents:

- Track cost and usage: read token and cost data from the message stream without an external backend.
- Hosting the Agent SDK: deploy agents in containers where you can set OpenTelemetry variables at the environment level.
- Monitoring: the complete reference for every environment variable, metric, and event the CLI emits.